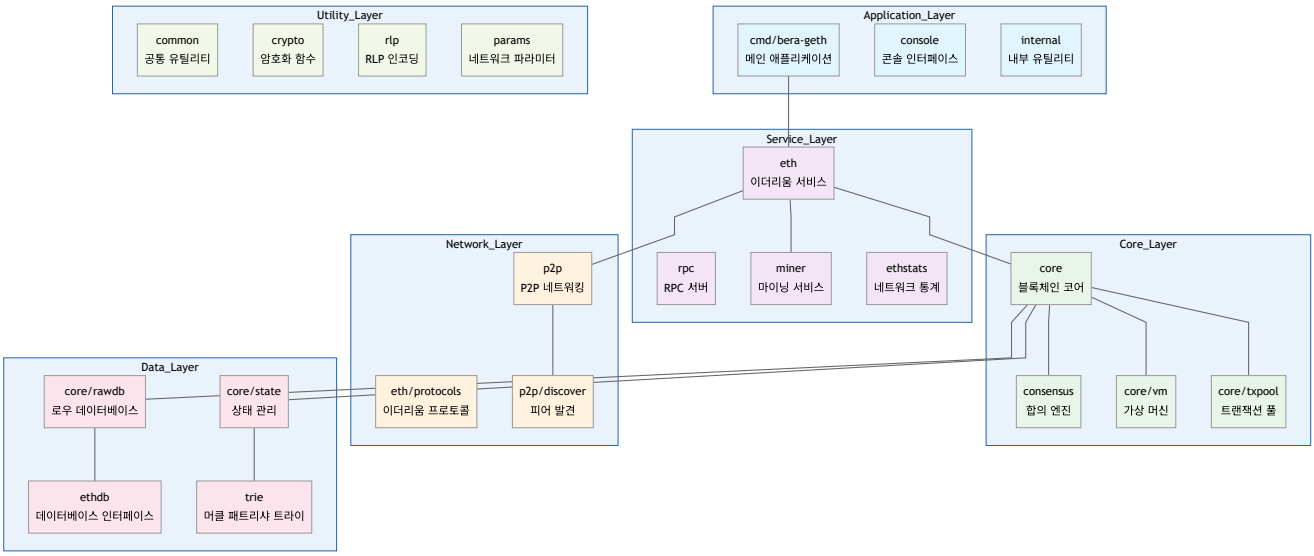
00 - Geth 디렉터리 구조 완전 가이드

개요

Go-Ethereum (Geth) 구현체는 복잡한 블록체인 시스템으로, 체계적인 디렉터리 구조를 통해 각 기능을 모듈화하고 있습니다. 본 가이드는 Geth의 모든 주요 디렉터리와 핵심 파일들의 역할을 상세히 설명하여, 개발자들이 코드베이스를 효율적으로 탐색할 수 있도록 돕습니다.

전체 아키텍처 다이어그램



주요 디렉터리 상세 분석

1. 애플리케이션 진입점 (Application Entry Points)

`cmd/` - 명령행 도구들

역할: 다양한 Geth 관련 실행 파일들의 메인 함수와 CLI 인터페이스

핵심 파일들:

- `cmd/bera-geth/main.go` - 메인 Geth 클라이언트 진입점
- `cmd/bera-geth/chaincmd.go` - 블록체인 관련 명령어 처리
- `cmd/bera-geth/consolecmd.go` - 콘솔 명령어 처리
- `cmd/utils/flags.go` - CLI 플래그 정의 및 처리
- `cmd/utils/cmd.go` - 공통 명령어 유틸리티

주요 하위 디렉터리:

- `cmd/abigen/` - ABI 바인딩 생성기
- `cmd/clef/` - 외부 서명자 도구
- `cmd/devp2p/` - P2P 네트워크 개발 도구
- `cmd/evm/` - EVM 테스트 도구

2. 이더리움 서비스 계층 (Ethereum Service Layer)

`eth/` - 이더리움 프로토콜 구현

역할: 이더리움 네트워크 프로토콜의 핵심 구현체

핵심 파일들:

- `eth/backend.go` - 이더리움 백엔드 서비스 (`eth.Ethereum` 구조체)
- `eth/handler.go` - 프로토콜 메시지 핸들러
- `eth/handler_eth.go` - ETH 프로토콜 핸들링
- `eth/handler_snap.go` - 스냅샷 프로토콜 핸들링
- `eth/api_backend.go` - API 백엔드 구현
- `eth/sync.go` - 블록체인 동기화 로직

주요 하위 디렉터리:

- `eth/downloader/` - 블록 다운로더 (`downloader.Downloader`)
- `eth/fetcher/` - 블록/트랜잭션 페처
- `eth/filters/` - 로그 필터링 시스템
- `eth/gasprice/` - 가스 가격 오라클
- `eth/protocols/` - 네트워크 프로토콜 구현
- `eth/tracers/` - 트랜잭션 추적기

`node/` - 노드 관리

역할: Geth 노드의 생명주기와 서비스 관리

핵심 파일들:

- `node/node.go` - 노드 구조체 (`node.Node`)와 생명주기 관리
- `node/config.go` - 노드 설정 관리
- `node/rpcstack.go` - RPC 스택 관리
- `node/api.go` - 노드 API 구현

3. 블록체인 코어 시스템 (Blockchain Core System)

`core/` - 블록체인 핵심 로직

역할: 블록체인의 핵심 기능인 블록 처리, 상태 관리, 트랜잭션 실행

핵심 파일들:

- `core/blockchain.go` - 블록체인 관리 (`core.BlockChain` 구조체)
- `InsertChain()` - 블록 체인 삽입
- `GetBlock()` - 블록 조회
- `CurrentBlock()` - 현재 블록 조회
- `core/state_processor.go` - 상태 처리기 (`core.StateProcessor`)

- `Process()` - 트랜잭션 처리
- `core/state_transition.go` - 상태 전환 로직
- `core/genesis.go` - 제네시스 블록 관리
- `core/headerchain.go` - 헤더 체인 관리
- `core/block_validator.go` - 블록 검증기

주요 하위 디렉터리:

`core/state/` - 상태 관리

- `core/state/statedb.go` - 상태 데이터베이스 (`state.StateDB`)
- `GetAccount()` - 계정 조회
- `SetBalance()` - 잔액 설정
- `Commit()` - 상태 커밋
- `core/state/state_object.go` - 상태 객체 관리
- `core/state/journal.go` - 상태 변경 저널

`core/types/` - 핵심 데이터 타입

- `core/types/block.go` - 블록 구조체 (`types.Block` , `types.Header`)
- `core/types/transaction.go` - 트랜잭션 구조체 (`types.Transaction`)
- `core/types/receipt.go` - 트랜잭션 영수증 (`types.Receipt`)
- `core/types/log.go` - 이벤트 로그 (`types.Log`)

`core/vm/` - 이더리움 가상 머신

- `core/vm/evm.go` - EVM 구조체 (`vm.EVM`)
- `NewEVM()` - EVM 생성자
- `Call()` - 컨트랙트 호출
- `Create()` - 컨트랙트 생성

- `core/vm/interpreter.go` - 바이트코드 해석기
- `core/vm/instructions.go` - 옵코드 구현
- `core/vm/gas_table.go` - 가스 비용 테이블

`core/txpool/` - 트랜잭션 풀

- `core/txpool/txpool.go` - 트랜잭션 풀 관리
- `core/txpool/legacypool/` - 레거시 트랜잭션 풀
- `core/txpool/blobpool/` - 블롭 트랜잭션 풀

`core/rawdb/` - 로우 데이터베이스

- `core/rawdb/database.go` - 데이터베이스 생성 및 관리
- `core/rawdb/accessors_chain.go` - 체인 데이터 접근자
- `core/rawdb/accessors_state.go` - 상태 데이터 접근자
- `core/rawdb/freezer.go` - 고대 데이터 저장소

4. 합의 시스템 (Consensus System)

`consensus/` - 합의 엔진

역할: 블록 검증과 합의 알고리즘 구현

핵심 파일들:

- `consensus/consensus.go` - 합의 엔진 인터페이스 (`consensus.Engine`)
- `VerifyHeader()` - 헤더 검증
- `Prepare()` - 블록 준비
- `Finalize()` - 블록 완료
- `consensus/errors.go` - 합의 관련 에러 정의

주요 하위 디렉터리:

- `consensus/beacon/` - 비콘 체인 합의 (`beacon.Beacon`)
- `consensus/clique/` - Clique PoA 합의
- `consensus/ethash/` - Ethash PoW 합의

`miner/` - 마이닝 시스템

역할: 블록 생성 및 마이닝 프로세스 관리

핵심 파일들:

- `miner/miner.go` - 마이너 관리 (`miner.Miner`)
- `miner/worker.go` - 마이닝 워커 (`miner.Worker`)
- `generateWork()` - 작업 생성
- `commitWork()` - 작업 커밋
- `miner/payload_building.go` - 페이로드 빌딩

5. 네트워킹 시스템 (Networking System)

`p2p/` - P2P 네트워킹

역할: 피어 투 피어 네트워크 통신 관리

핵심 파일들:

- `p2p/server.go` - P2P 서버 (`p2p.Server`)
- `Start()` - 서버 시작
- `Stop()` - 서버 중단
- `AddPeer()` - 피어 추가
- `p2p/peer.go` - 피어 관리 (`p2p.Peer`)
- `p2p/protocol.go` - 프로토콜 정의
- `p2p/dial.go` - 피어 연결 관리

주요 하위 디렉터리:

- `p2p/discover/` - 피어 발견 시스템
- `p2p/discover/v4.go` - UDPv4 발견 (`discover.UDPv4`)
- `p2p/discover/v5.go` - UDPv5 발견 (`discover.UDPv5`)
- `p2p/enode/` - 이더리움 노드 식별자
- `p2p/enr/` - 이더리움 노드 레코드
- `p2p/nat/` - NAT 순회
- `p2p/netutil/` - 네트워크 유틸리티

6. 데이터 저장 시스템 (Data Storage System)

`ethdb/` - 데이터베이스 인터페이스

역할: 데이터베이스 추상화 레이어

핵심 파일들:

- `ethdb/database.go` - 데이터베이스 인터페이스 (`ethdb.Database`)
- `ethdb/batch.go` - 배치 연산 인터페이스
- `ethdb/iterator.go` - 반복자 인터페이스

주요 하위 디렉터리:

- `ethdb/leveldb/` - LevelDB 구현
- `ethdb/pebble/` - PebbleDB 구현
- `ethdb/memorydb/` - 메모리 데이터베이스

`trie/` - 머클 패트리샤 트라이

역할: 이더리움의 핵심 데이터 구조인 트라이 구현

핵심 파일들:

- `trie/trie.go` - 트라이 구조체 (`trie.Trie`)
- `Get()` - 값 조회
- `Update()` - 값 업데이트
- `Delete()` - 값 삭제
- `trie/secure_trie.go` - 보안 트라이 (`trie.SecureTrie`)
- `trie/database.go` - 트라이 데이터베이스
- `trie/sync.go` - 트라이 동기화

7. RPC 및 API 시스템 (RPC & API System)

`rpc/` - RPC 서버

역할: JSON-RPC 서버 구현 및 클라이언트 통신

핵심 파일들:

- `rpc/server.go` - RPC 서버 (`rpc.Server`)
- `Start()` - 서버 시작
- `RegisterName()` - API 등록
- `rpc/client.go` - RPC 클라이언트 (`rpc.Client`)
- `rpc/http.go` - HTTP 전송 계층
- `rpc/websocket.go` - WebSocket 전송 계층
- `rpc/ipc.go` - IPC 전송 계층

`internal/ethapi/` - 이더리움 API

역할: 이더리움 관련 RPC API 구현

핵심 파일들:

- `internal/ethapi/api.go` - 공개 API (`ethapi.PublicEthereumAPI`)

- `internal/ethapi/backend.go` - API 백엔드 인터페이스
- `internal/ethapi/transaction_args.go` - 트랜잭션 인수 처리

8. 유틸리티 및 지원 시스템 (Utility & Support Systems)

`common/` - 공통 유틸리티

역할: 전체 시스템에서 사용되는 공통 타입과 함수

핵심 파일들:

- `common/types.go` - 기본 타입 (`common.Address`, `common.Hash`)
- `common/bytes.go` - 바이트 조작 함수
- `common/math/` - 수학 유틸리티
- `common/hexutil/` - 16진수 인코딩/디코딩

`crypto/` - 암호화 함수

역할: 암호화 관련 함수와 알고리즘

핵심 파일들:

- `crypto/crypto.go` - 핵심 암호화 함수
- `Keccak256()` - Keccak-256 해시
- `Sign()` - 디지털 서명
- `Ecrecover()` - 서명 복구
- `crypto/secp256k1/` - secp256k1 타원곡선 암호화

`rlp/` - RLP 인코딩

역할: Recursive Length Prefix 인코딩/디코딩

핵심 파일들:

- `rlp/encode.go` - RLP 인코딩
- `rlp/decode.go` - RLP 디코딩

`params/` - 네트워크 파라미터

역할: 네트워크 설정과 하드포크 파라미터

핵심 파일들:

- `params/config.go` - 체인 설정 (`params.ChainConfig`)
- `params/protocol_params.go` - 프로토콜 상수
- `params/bootnodes.go` - 부트노드 목록

9. 클라이언트 라이브러리 (Client Libraries)

`ethclient/` - Go 클라이언트

역할: Geth와 상호작용하기 위한 Go 클라이언트 라이브러리

핵심 파일들:

- `ethclient/ethclient.go` - 이더리움 클라이언트 (`ethclient.Client`)
- `Dial()` - 클라이언트 연결
- `BlockByNumber()` - 블록 조회
- `SendTransaction()` - 트랜잭션 전송

`accounts/` - 계정 관리

역할: 계정과 지갑 관리

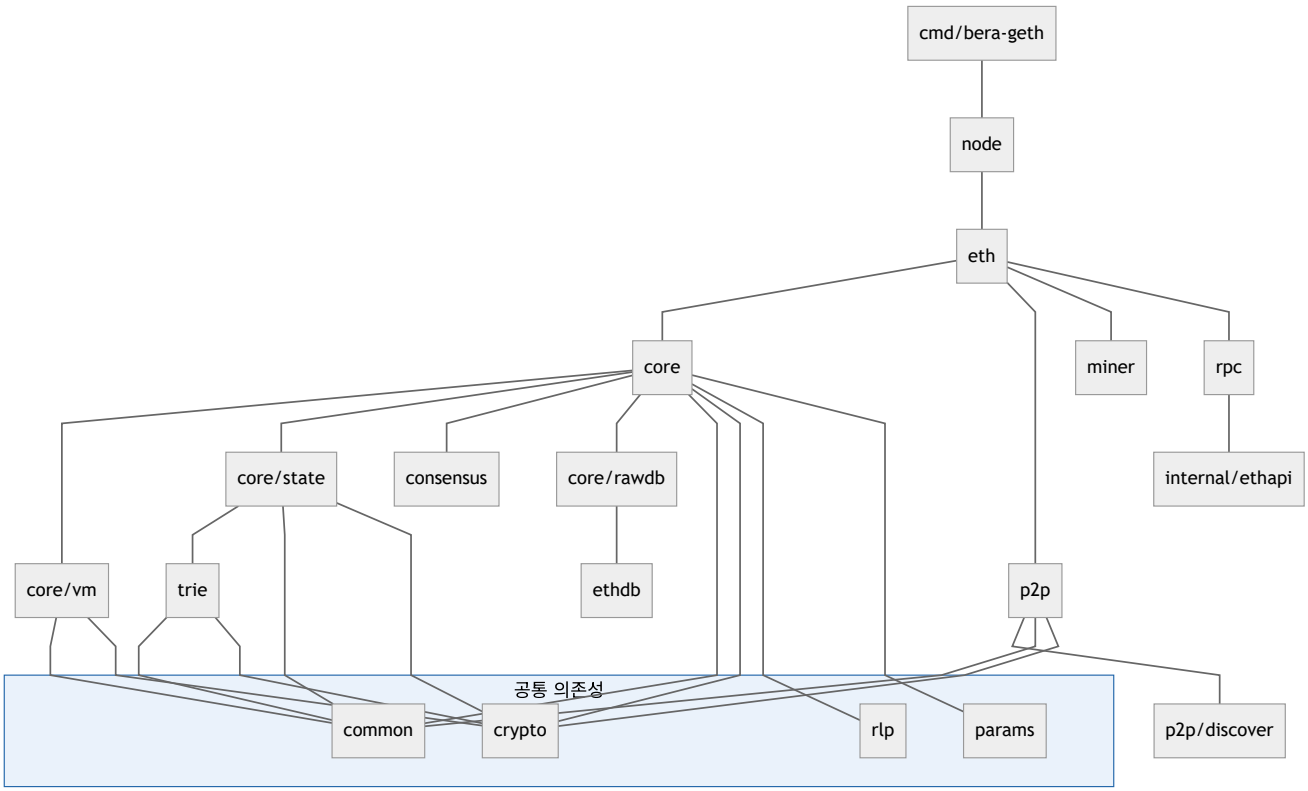
핵심 파일들:

- `accounts/manager.go` - 계정 관리자 (`accounts.Manager`)
- `accounts/keystore/` - 키스토어 구현

- `accounts/usbwallet/` - USB 하드웨어 지갑

디렉터리 간 의존성 관계

핵심 의존성 흐름



계층별 의존성 규칙

1. **상위 계층 → 하위 계층**: 허용

2. **하위 계층 → 상위 계층**: 금지 (순환 의존성 방지)

3. **동일 계층 내**: 제한적 허용 (인터페이스 통해)

코드 분석 학습 경로

초보자를 위한 단계별 학습 순서

1단계: 기본 구조 이해 (1-2주)

```
1. common/ - 기본 타입과 유틸리티 함수 이해
2. crypto/ - 암호화 기본 개념 학습
3. rlp/ - 데이터 직렬화 방식 이해
4. params/ - 네트워크 파라미터 확인
```

2단계: 데이터 구조 학습 (2-3주)

```
1. core/types/ - 블록, 트랜잭션, 영수증 구조 이해
2. trie/ - 머클 패트리샤 트라이 구조 학습
3. ethdb/ - 데이터베이스 인터페이스 이해
4. core/rawdb/ - 실제 데이터 저장 방식 학습
```

3단계: 핵심 로직 이해 (3-4주)

```
1. core/blockchain.go - 블록체인 관리 로직
2. core/state/ - 상태 관리 시스템
3. core/vm/ - EVM 실행 엔진
```

4. consensus/ - 합의 메커니즘

4단계: 네트워킹 시스템 (2-3주)

1. p2p/server.go - P2P 서버 구조
2. p2p/discover/ - 피어 발견 메커니즘
3. eth/handler.go - 프로토콜 핸들링
4. eth/downloader/ - 블록 동기화

5단계: 서비스 계층 (2-3주)

1. eth/backend.go - 이더리움 서비스
2. miner/ - 마이닝 시스템
3. core/txpool/ - 트랜잭션 풀
4. rpc/ - RPC 서버

6단계: 애플리케이션 계층 (1-2주)

1. node/ - 노드 관리
2. cmd/bera-geth/ - 메인 애플리케이션
3. internal/ethapi/ - API 구현
4. ethclient/ - 클라이언트 라이브러리

기능별 탐색 가이드

블록 처리 과정을 이해하고 싶다면:

```
1. core/blockchain.go - InsertChain() 메서드
2. core/block_validator.go - ValidateBody() 메서드
3. consensus/ - 합의 엔진 구현
4. core/state_processor.go - Process() 메서드
5. core/vm/ - 트랜잭션 실행
```

트랜잭션 생명주기를 추적하고 싶다면:

```
1. core/txpool/ - 트랜잭션 풀 관리
2. eth/handler.go - 트랜잭션 수신
3. core/state_transition.go - 상태 전환
4. core/vm/evm.go - 실행
5. core/types/receipt.go - 영수증 생성
```

P2P 네트워킹을 이해하고 싶다면:

```
1. p2p/server.go - 서버 시작과 피어 관리
2. p2p/discover/ - 피어 발견
3. eth/handler.go - 프로토콜 메시지 처리
4. eth/protocols/ - 구체적인 프로토콜 구현
```

데이터 저장을 이해하고 싶다면:

1. ethdb/ - 데이터베이스 인터페이스
2. core/rawdb/ - 로우 데이터 접근
3. trie/ - 상태 트라이 구조
4. core/state/ - 상태 관리

디버깅 및 로그 분석 가이드

로그 메시지로 관련 디렉터리 찾기

블록 관련 에러

- "Invalid block" → core/block_validator.go
- "Bad block" → core/blockchain.go
- "Chain reorg" → core/blockchain.go

트랜잭션 관련 에러

- "Transaction pool" → core/txpool/
- "Invalid transaction" → core/types/transaction.go
- "Gas limit exceeded" → core/vm/

네트워크 관련 에러

- "Peer disconnected" → p2p/peer.go
- "Protocol handshake" → eth/handler.go
- "Discovery" → p2p/discover/

합의 관련 에러

- `"Consensus"` → `consensus/`
- `"Invalid seal"` → `consensus/` 하위 구현체
- `"Fork detected"` → `core/forkid/`

성능 모니터링 포인트

메트릭 수집 위치

- `metrics/` - 메트릭 시스템
- `core/blockchain.go` - 블록 처리 성능
- `p2p/server.go` - 네트워크 성능
- `core/txpool/` - 트랜잭션 풀 성능

한글 기술 용어 사전

블록체인 핵심 용어

- **Block**: 블록
- **Transaction**: 트랜잭션 (거래)
- **Receipt**: 영수증
- **State**: 상태
- **Trie**: 트라이 (트리 자료구조)
- **Hash**: 해시
- **Address**: 주소
- **Account**: 계정
- **Balance**: 잔액
- **Nonce**: 논스 (일회용 번호)

네트워킹 용어

- **Peer**: 피어 (동등 노드)
- **Node**: 노드
- **Protocol**: 프로토콜
- **Handshake**: 핸드셰이크 (연결 설정)
- **Discovery**: 발견
- **Sync**: 동기화
- **Downloader**: 다운로더

합의 및 마이닝 용어

- **Consensus**: 합의
- **Mining**: 마이닝 (채굴)
- **Miner**: 마이너 (채굴자)
- **Validator**: 검증자
- **Seal**: 봉인
- **Difficulty**: 난이도
- **Gas**: 가스
- **Gas Limit**: 가스 한도
- **Gas Price**: 가스 가격

EVM 관련 용어

- **Virtual Machine**: 가상 머신
- **Bytecode**: 바이트코드

- **Opcode**: 옵코드 (연산 코드)
- **Stack**: 스택
- **Memory**: 메모리
- **Storage**: 저장소
- **Contract**: 컨트랙트 (계약)
- **Call**: 호출
- **Create**: 생성

데이터베이스 용어

- **Database**: 데이터베이스
- **Batch**: 배치
- **Iterator**: 반복자
- **Snapshot**: 스냅샷
- **Commit**: 커밋
- **Rollback**: 롤백

문서 템플릿

컴포넌트 분석 문서 템플릿

```
# [Index] - [컴포넌트명] 시스템


## 개요

- 컴포넌트의 역할과 중요성
```

```
- 전체 시스템에서의 위치
- 주요 책임사항


## 핵심 구조체 및 인터페이스
### [구조체명]
```go
type StructName struct {
// 주요 필드들
}
```
```

- 구조체 설명
- 주요 메서드 목록

주요 프로세스

[프로세스명]

1. 단계별 설명
2. 관련 함수 호출
3. 데이터 흐름

코드 예시

[기능명] 사용 예시

```
// 실제 코드 예시
```

최적화 및 성능

- 성능 고려사항
- 최적화 기법
- 메트릭 수집

디버깅 및 모니터링

- 주요 로그 포인트
- 에러 처리 방식
- 모니터링 메트릭

관련 컴포넌트

- 의존성 관계
- 상호작용 방식
- 연관 문서 링크

이 가이드를 통해 Geth 코드베이스를 체계적으로 탐색하고 이해할 수 있습니다. 각 디렉터리의 역